

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I **K.MAHESH BABU(192525122)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **K.MAHESH BABU(192525122)**

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized	Most constraints identified	Some constraints identified	Constraints poorly understood

		accurately			
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

Answer

HDFS Storage Design for High-Availability Media Platform

Identification and Prioritization of Constraints

To meet the specific requirements of the media platform, the design must balance reliability with fiscal responsibility:

- **Availability (>99.9%):** This is the primary constraint. In HDFS, this necessitates a **High Availability (HA) Cluster** configuration to eliminate the NameNode as a Single Point of Failure (SPOF).
- **Replication Factor (≤ 3):** This constraint limits data redundancy. To achieve high availability without exceeding this, the design must utilize **Rack Awareness** to ensure block placement is distributed across different physical failure domains.
- **Cost Efficiency:** As media files (video/audio) are large, standard triplication is expensive. The design must incorporate **HDFS Erasure Coding (EC)** for warm/cold data and **Heterogeneous Storage Management (HSM)** to optimize hardware costs.

Proposed System Architecture

1. High Availability (HA) Framework

- **Active/Standby NameNodes:** Deploy two NameNodes (Active and Standby) to ensure the metadata service remains available.
- **Quorum Journal Manager (QJM):** Use a group of JournalNodes (minimum 3) to synchronize edit logs between NameNodes, ensuring zero data loss during failover.
- **Zookeeper Failover Controller (ZKFC):** Automate the transition from Standby to Active if the primary node fails, maintaining the >99.9% uptime requirement.

2. Data Placement and Rack Awareness

- **Policy:** Configure a rack-aware topology where the first replica is on the local node, the second on a different node in the same rack, and the third on a node in a different rack.
- **Impact:** This satisfies the replication factor of 3 while protecting against both individual disk failure and top-of-rack (ToR) switch failure, significantly boosting availability.

3. Storage Optimization for Cost Efficiency

- **Erasure Coding (EC):** For older media content that is accessed less frequently, transition from 3x replication to an **RS-6-3 policy** (Reed-Solomon). This reduces storage overhead from 200% to roughly 50% while maintaining higher fault tolerance than standard replication.
- **Heterogeneous Storage (Tiering):**
- **SSD/RAM_DISK:** For trending/newly uploaded media (Hot data).
- **DISK (SATA):** For standard library content (Warm data).
- **ARCHIVE (High-density HDD):** For legacy content (Cold data).

Technical Justification

- **Availability:** The HA NameNode configuration ensures that the control plane remains operational even during hardware maintenance or crashes, meeting the 99.9% SLA.

- **Constraint Adherence:** By utilizing Rack Awareness, we maximize the utility of the 3x replication limit, ensuring that a single rack failure does not result in data loss.
- **Cost-Benefit:** Erasure Coding is the critical driver for cost efficiency. In a media platform where data grows exponentially, EC allows the platform to store twice as much data on the same hardware compared to traditional replication, without violating the replication cap for active writes.

Figure: High Availability HDFS Design for Media Platform

- Active NameNode
- Standby NameNode
- Quorum Journal Manager (QJM)
- Zookeeper Service
- DataNode (Hot Tier - SSD)
- DataNode (Cold Tier - HDD)
- Rack 1 (Replica 1)
- Rack 2 (Replica 2 & 3)

Active NameNode -> Quorum Journal Manager (QJM) (Edit Logs)

Standby NameNode -> Quorum Journal Manager (QJM) (Sync Logs)

Zookeeper Service -> Active NameNode (Health Check)

Active NameNode -> DataNode (Hot Tier - SSD) (Block Report)

Active NameNode -> DataNode (Cold Tier - HDD) (Block Report)

DataNode (Hot Tier - SSD) -> Rack 1 (Replica 1) (Placement)

DataNode (Cold Tier - HDD) -> Rack 2 (Replica 2 & 3) (Placement)

Question 2

A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

Answer

Proposal for Optimized MapReduce Job Design

To process log data within a strict **20-minute batch window** on a resource-constrained cluster, the job design must prioritize data locality, minimize network I/O, and optimize task scheduling via YARN.

1. Identification and Prioritization of Constraints

- **Temporal Constraint (High Priority):** The 20-minute window necessitates aggressive parallelization and the elimination of bottlenecks in the Shuffle and Sort phase.
- **Resource Constraint (High Priority):** Limited cluster nodes require efficient memory management to avoid swapping and to ensure high CPU utilization.
- **Data Volume (Medium Priority):** Log data is typically semi-structured and repetitive, offering opportunities for significant compression.

2. Proposed Job Design Architecture

- **Input Format and Splitting:** Use `CombineFileInputFormat` to merge small log files into larger splits. This reduces the number of Map tasks, decreasing the overhead of JVM startups on a limited node cluster.

- **Data Compression:** Implement **Snappy** or **LZO** compression for intermediate Map outputs. These codecs offer a high speed-to-compression ratio, reducing disk I/O and network congestion during the shuffle phase without taxing the CPU excessively.
- **Combiner Function:** A `Combiner` (mini-reducer) must be implemented to perform local aggregation on the Map side. This drastically reduces the volume of data transferred across the network to the Reducers.
- **Custom Partitioning:** Use a `HashPartitioner` or a custom partitioner based on log timestamps to ensure an even distribution of data across Reducers, preventing "skewed tasks" that could break the 20-minute deadline.

3. Task Distribution and YARN Management

- **Data Locality:** YARN will be configured to prioritize **NODE_LOCAL** tasks. By scheduling Map tasks on the same nodes where HDFS blocks reside, we eliminate the need for data transfer during the initial phase.
- **Container Sizing:** Given the limited nodes, `yarn.scheduler.maximum-allocation-mb` should be tuned to allow multiple containers per node, ensuring that if one task is I/O bound, another can utilize the CPU.
- **Speculative Execution:** Disable speculative execution if the cluster is highly resource-constrained, as the cost of running duplicate tasks may outweigh the benefit of catching a single slow node within such a short window.

4. Technical Justification

- **Efficiency:** The use of a **Combiner** is the most critical factor here; by aggregating log entries (e.g., counting error codes) locally, we reduce the shuffle payload by up to 90%, ensuring the 20-minute limit is met.
- **Resource Utilization:** `CombineFileInputFormat` prevents "Map Task Overload," where the cluster spends more time managing task lifecycles than processing data.
- **Reliability:** Snappy compression provides the best balance for real-time batch processing, as it is non-splittable but extremely fast, fitting the requirement for rapid throughput over maximum storage savings.

Question 3

A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

Answer

Design of a YARN-Managed Multi-Tenant Scheduler Policy

To address the diverse workloads of ETL, reporting, and machine learning (ML) while ensuring fairness and reliability, a **Capacity Scheduler** with hierarchical queues and preemption is the optimal design choice. This configuration balances strict SLAs for ETL with the resource-intensive nature of ML.

1. Identification and Prioritization of Constraints

- **Priority for ETL (Highest):** ETL processes are time-sensitive and serve as the foundation for downstream reporting. They require guaranteed resources and minimal latency.
- **Fairness (Medium):** Reporting and ML jobs must not be starved by ETL. Resources should be distributed equitably among users within their respective pools.

- **Recovery from Node Failure (High):** The system must ensure job continuity. YARN's ApplicationMaster (AM) restart and container rescheduling mechanisms must be tuned to handle hardware volatility without manual intervention.

2. Hierarchical Queue Design

A root-level partitioning strategy is employed to isolate workloads while allowing for resource elasticity:

- **ETL Queue (40% Capacity):** Configured with high **User Limit Factor** and **Priority**. This queue is allowed to expand beyond its capacity if other queues are idle but has the highest scheduling priority.
- **Reporting Queue (30% Capacity):** Optimized for short, concurrent queries. Uses **Intra-queue Fairness** to ensure multiple analysts can run reports simultaneously.
- **ML Queue (30% Capacity):** Designed for long-running, resource-heavy jobs. This queue uses **GPU/FPGA resource scheduling** if available and is the primary candidate for preemption.

3. Application of YARN Concepts

- **Capacity Scheduler with Preemption:** To satisfy the "Priority for ETL" constraint, **Preemption** is enabled. If the ETL queue requires resources currently held by the ML queue (beyond its guaranteed capacity), YARN will gracefully kill ML containers to reclaim resources for ETL.
- **Node Labels:** Assigning specific labels (e.g., `high-mem` for ML, `high-io` for ETL) ensures that jobs are mapped to nodes best suited for their specific constraints, improving overall cluster efficiency.
- **Resource Calculator:** The `DominantResourceCalculator` (DRC) must be used instead of the default `DefaultResourceCalculator`. Since ML jobs are often memory-heavy and ETL is CPU-heavy, DRC ensures fairness across multiple resource dimensions (CPU and RAM).

4. Fault Tolerance and Recovery Mechanisms

To address node failure recovery, the following configurations are implemented:

- **AM Restart Policy:** The `yarn.resourcemanager.am.max-attempts` is set to a minimum of 2. If a node hosting the ApplicationMaster fails, YARN automatically restarts the AM on a healthy node.
- **Work-Preserving Restart:** Enabled via `yarn.resourcemanager.work-preserving-recovery.enabled`. This allows the ResourceManager to reconstruct the state of the cluster by polling NodeManagers, ensuring that running containers are not killed if the RM itself fails.
- **Timeline Service V2:** Implemented to track the lifecycle of ETL flows, providing a centralized audit trail to identify where a failure occurred during the pipeline.

5. Technical Justification

Choosing the **Capacity Scheduler** over the Fair Scheduler is justified by the requirement for "Priority for ETL." While Fair Scheduling aims for equality, Capacity Scheduling allows for "guaranteed minimums" which is essential for production ETL pipelines. The inclusion of **Preemption** mitigates the risk of long-running ML jobs "locking" the cluster, ensuring that the fairness constraint does not compromise the high-priority ETL SLA. Finally, the **Work-Preserving Restart** configuration ensures that node failures result in localized task retries rather than total job failures, maintaining system robustness.

Figure: YARN Scheduler Architecture for Multi-Workload Constraints

- YARN ResourceManager
- Capacity Scheduler (Preemption ON)
- ETL Queue (High Priority)
- Reporting Queue (Fairness)
- ML Queue (Best Effort)
- NodeManagers (Recovery Enabled)

YARN ResourceManager -> Capacity Scheduler (Preemption ON) (Policy Enforcement)
Capacity Scheduler (Preemption ON) -> ETL Queue (High Priority) (40% Guaranteed)
Capacity Scheduler (Preemption ON) -> Reporting Queue (Fairness) (30% Guaranteed)
Capacity Scheduler (Preemption ON) -> ML Queue (Best Effort) (30% Guaranteed)
ETL Queue (High Priority) -> NodeManagers (Recovery Enabled) (Preempts Q3)
NodeManagers (Recovery Enabled) -> YARN ResourceManager (Heartbeat/Recovery)

Question 4

An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

Answer

Proposed Architecture for Heterogeneous Data Ingestion

To address the ingestion of data from **Storage Area Networks (SAN)**, **Network Attached Storage (NAS)**, and **Operational Databases (RDBMS)** while ensuring reliability and minimal duplication, the following constrained architecture is proposed. This design leverages the Hadoop ecosystem to centralize disparate data silos into a unified **Data Lake**.

1. Data Ingestion Layer (The Entry Point)

- **Apache Sqoop (for RDBMS):** Used specifically for structured data in operational databases. It utilizes connector-based ingestion to transfer data between Hadoop and relational databases.
- ***Constraint Handling:*** It supports incremental imports using check-columns to ensure only new or modified rows are ingested, directly addressing the **minimal duplication** requirement.
- **Apache Flume / NFS Gateway (for NAS/SAN):** For file-based storage like NAS and SAN, Flume agents or the HDFS NFS Gateway are utilized.
- ***Constraint Handling:*** Flume provides "at-least-once" delivery semantics and multi-hop flows, ensuring **backup reliability** during the transit of unstructured or semi-structured files.

2. Storage Layer (The Core)

- **Hadoop Distributed File System (HDFS):** Acts as the primary landing zone.
- ***Constraint Handling:*** HDFS inherently provides reliability through its **Replication Factor** (typically 3). By distributing blocks across different DataNodes, the system ensures that hardware failures in the SAN/NAS source do not result in data loss within the Big Data platform.

3. Resource Management and Processing

- **YARN (Yet Another Resource Negotiator):** Manages the computational resources for the ingestion jobs.
- ***Constraint Handling:*** YARN allows for **Resource Quotas**, ensuring that heavy ingestion tasks from the SAN do not starve the operational database migration tasks, maintaining system stability.

4. Data Integrity and De-duplication Logic

- **HBase / Hive Acid Tables:** Once data lands in HDFS, a compaction or reconciliation layer using Hive (with ACID properties) or HBase is employed.

- ***Constraint Handling:*** By using **Upsert** logic (Update + Insert), the architecture identifies records that already exist based on primary keys or file hashes, effectively purging duplicates that may have been ingested during overlapping backup windows.

Justification of Components

- **Scalability:** The architecture is horizontally scalable. As SAN/NAS capacity grows, HDFS DataNodes can be added without reconfiguring the ingestion logic.
- **Reliability:** The use of Flume's channel-based buffering ensures that if the destination HDFS is temporarily unavailable, data is staged locally rather than lost.
- **Efficiency:** By utilizing Sqoop's incremental mode and Hive's de-duplication queries, the system minimizes the storage footprint, adhering to the constraint of **minimal duplication**.
- **Interoperability:** The combination of Sqoop (structured) and Flume (unstructured) provides a comprehensive coverage for the enterprise's heterogeneous source environment.

Figure: Constrained Data Ingestion Architecture

- Operational DBs (RDBMS)
- SAN / NAS (File Storage)
- Apache Sqoop (Incremental)
- Apache Flume / NFS Gateway
- HDFS (Replicated Storage)
- Hive ACID (De-duplication)
- YARN Resource Manager

Operational DBs (RDBMS) -> Apache Sqoop (Incremental) (JDBC)

SAN / NAS (File Storage) -> Apache Flume / NFS Gateway (File Stream)

Apache Sqoop (Incremental) -> HDFS (Replicated Storage) (Structured Data)

Apache Flume / NFS Gateway -> HDFS (Replicated Storage) (Unstructured Data)

HDFS (Replicated Storage) -> Hive ACID (De-duplication) (Refinement)

YARN Resource Manager -> Apache Sqoop (Incremental) (Resource Alloc)

YARN Resource Manager -> Apache Flume / NFS Gateway (Resource Alloc)

Question 5

Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Answer

End-to-End Hadoop Ecosystem Design for Secure, High-Availability Data Pipelines

To address the requirements of secure data movement and high availability (HA) while integrating with Apache NiFi, the following architecture leverages the core Hadoop stack augmented by enterprise-grade security and orchestration tools.

1. Identification and Prioritization of Constraints

- **Constraint A: Secure Data Movement:** Requires encryption in transit (TLS/SSL), encryption at rest (HDFS Transparent Encryption), and robust authentication (Kerberos).

- **Constraint B: High Availability (HA):** Elimination of single points of failure (SPOFs) in the NameNode and ResourceManager, utilizing Quorum Journal Manager (QJM).
- **Constraint C: Integration with Apache NiFi:** Seamless ingestion from diverse sources into HDFS/Hive while maintaining data provenance.

2. System Design and Component Integration

Data Ingestion Layer (Apache NiFi)

Apache NiFi acts as the entry point, utilizing **Site-to-Site (S2S)** protocols for secure data transfer. To satisfy the security constraint, NiFi is integrated with **Apache Ranger** for fine-grained access control and **Apache Atlas** for metadata and lineage tracking. Sensitive data is masked or encrypted at the edge before hitting the Hadoop cluster.

Storage Layer (HDFS with HA)

- **High Availability:** The design employs two NameNodes (Active and Standby). The **Quorum Journal Manager (QJM)** ensures that edit logs are synchronized across JournalNodes, preventing data loss during failover.
- **Security:** HDFS is configured with **Transparent Data Encryption (TDE)**. A Key Management Server (KMS) manages the zone keys, ensuring that even if physical disks are compromised, the data remains unreadable.

Processing and Resource Management (YARN & MapReduce/Spark)

- **YARN HA:** Multiple ResourceManagers are deployed; if the active instance fails, the standby takes over using Zookeeper for leader election.
- **Authentication:** The entire cluster is secured via **Kerberos**, requiring valid tickets for any job submission or data access, preventing unauthorized "rogue" nodes from joining the cluster.

3. Justification of Design Choices

- **Integration Strategy:** NiFi is chosen over traditional ETL because of its "Flow-Based Programming" model, which handles back-pressure and data buffering, ensuring the Hadoop cluster is not overwhelmed during peak ingestion.
- **Security Enforcement:** The combination of **Kerberos** (Authentication) and **Apache Ranger** (Authorization) creates a zero-trust environment. Ranger provides a centralized console to manage security policies across HDFS, Hive, and HBase.
- **Availability Logic:** By utilizing **Zookeeper Failover Controller (ZKFC)**, the system can detect NameNode failures in real-time and trigger an automatic failover, maintaining a 99.99% uptime requirement for enterprise analytics.

4. Data Flow Summary

1. NiFi ingests raw data via encrypted HTTPS/S2S.
2. Data is pushed to **HDFS Landing Zone** using the 'PutHDFS' processor with Kerberos credentials.
3. **Apache Ranger** validates the write request.
4. **YARN** allocates resources for a **MapReduce/Spark** job to transform the data.
5. Final processed data is stored in **Apache Hive** for BI consumption, with all metadata captured by **Apache Atlas** for compliance.

Figure: Secure HA Hadoop Ecosystem Architecture

- Data Sources (IoT/DB/Logs)
- Apache NiFi (Secure Ingestion)
- Kerberos / Ranger (Security)

- Active NameNode (HA)
- Standby NameNode (QJM)
- HDFS (Encrypted Storage)
- YARN Resource Manager (HA)
- Data Nodes / Processing

Data Sources (IoT/DB/Logs) -> Apache NiFi (Secure Ingestion) (TLS Encrypted)

Apache NiFi (Secure Ingestion) -> Kerberos / Ranger (Security) (Auth Check)

Apache NiFi (Secure Ingestion) -> Active NameNode (HA) (Write Request)

Active NameNode (HA) -> Standby NameNode (QJM) (Edit Log Sync)

Active NameNode (HA) -> HDFS (Encrypted Storage) (Metadata)

YARN Resource Manager (HA) -> Data Nodes / Processing (Task Allocation)

HDFS (Encrypted Storage) -> Data Nodes / Processing (Data Blocks)